

Credit Risk & Anomaly Detection

Rapport technique et métier du projet

Scoring de risque de crédit (classification supervisée) et détection de transactions anormales (apprentissage non supervisé) sur données financières réelles

Pipeline : SQL → Python → Feature Engineering → Imbalanced Learning
→ XGBoost → SHAP → Isolation Forest / DBSCAN

*Auteur : Darryll Joseph — Projet portfolio
Septembre 2026*

Sommaire

1. Résumé exécutif
2. Contexte métier et objectifs
3. Données et sources
4. Architecture du pipeline
5. Volet A — Credit Risk (classification supervisée)
 - 5.1 Exploration SQL
 - 5.2 Analyse exploratoire (EDA)
 - 5.3 Feature engineering
 - 5.4 Gestion du déséquilibre de classes
 - 5.5 Entraînement XGBoost
 - 5.6 Évaluation détaillée sur le test set
 - 5.7 Interprétabilité (SHAP)
6. Volet B — Anomaly Detection (non supervisé)
 - 6.1 Analyse exploratoire (EDA)
 - 6.2 Isolation Forest
 - 6.3 DBSCAN
 - 6.4 Comparaison et visualisation
7. Discussion, limites et recommandations
8. Annexes — stack technique et reproductibilité

1. Résumé exécutif

Ce projet implémente, de bout en bout, deux cas d'usage complémentaires du risque financier : (A) la prédiction du risque de défaut d'un emprunteur au moment de la demande de crédit, et (B) la détection de transactions anormales potentiellement frauduleuses en production. Les deux volets s'appuient sur des données réelles et publiques (aucune donnée synthétique) et couvrent l'ensemble de la chaîne data science : extraction et agrégation SQL, feature engineering, gestion du déséquilibre de classes, modélisation (XGBoost), évaluation rigoureuse, interprétabilité (SHAP) et détection d'anomalies non supervisée (Isolation Forest, DBSCAN).

Résultats clés

- **Volet A** : modèle XGBoost avec pondération de classe (*scale_pos_weight*) retenu après comparaison rigoureuse de 3 stratégies. ROC-AUC = 0.783 sur test set, F1 = 0.337 au seuil de décision optimisé (0.65).
- **Découverte contre-intuitive documentée** : SMOTE, pourtant la technique de rééquilibrage la plus connue, performe moins bien que la simple pondération de classe sur ce dataset à 222 dimensions — un résultat réel et pédagogique, pas un cas d'école.
- **Volet B** : Isolation Forest appliqué à l'intégralité des 284 807 transactions (F1 = 0.256) ; DBSCAN sur échantillon stratifié après correction d'un problème de sélection d'hyperparamètre (F1 = 0.428 après correction, contre un échec quasi total avant correction).
- **Interprétabilité** : SHAP confirme que les features construites via agrégation SQL (retards de paiement, taux de refus historique, utilisation de carte de crédit) contribuent significativement à la prédiction, aux côtés des scores externes fournis nativement dans les données.

Ce que ce rapport documente

Au-delà des résultats, ce document explique **pourquoi** chaque choix méthodologique a été fait — notamment pourquoi tel modèle a été retenu comme "meilleur" plutôt qu'un autre — et fournit une lecture détaillée de chacune des 23 visualisations produites pendant le projet, à la fois sur le plan technique (ce que montre le graphique) et métier (ce que cela signifie pour la décision de crédit ou la lutte antifraude).

2. Contexte métier et objectifs

Le scénario métier retenu est celui d'une institution de crédit (type fintech ou établissement bancaire) confrontée à deux problématiques distinctes mais complémentaires dans la gestion du risque financier :

Problématique A — Octroi de crédit

Avant d'accorder un prêt, l'établissement doit évaluer la probabilité qu'un emprunteur fasse défaut (ne rembourse pas). Une mauvaise estimation coûte cher dans les deux sens : accorder un crédit à un client à haut risque génère des pertes directes (le capital prêté n'est pas recouvré), tandis que refuser un crédit à un bon client fait perdre un revenu d'intérêts et un client potentiellement fidèle. L'enjeu est donc de produire un score de risque fiable et **interprétable** — la réglementation bancaire (ex. exigences de type "droit à l'explication") impose souvent de justifier une décision de refus.

Problématique B — Surveillance des transactions

Une fois les comptes ouverts et les cartes actives, l'établissement doit surveiller les transactions en continu pour détecter des comportements anormaux évocateurs de fraude (carte volée, usurpation, transaction inhabituelle). Contrairement au risque de crédit, on ne dispose généralement pas d'un historique de labels fiable et exhaustif en production — d'où l'intérêt d'approches **non supervisées** capables de signaler des transactions suspectes sans avoir été explicitement entraînées sur des exemples de fraude.

Pourquoi traiter les deux volets dans un même projet

Bien que techniquement différents (classification supervisée déséquilibrée vs détection d'anomalies non supervisée), les deux volets partagent la même finalité métier — quantifier et gérer le risque financier — et la même contrainte structurelle : dans les deux cas, l'événement d'intérêt (défaut, fraude) est rare. Ce projet permet ainsi de démontrer la maîtrise des deux grandes familles de techniques utilisées en gestion du risque financier, sur un pipeline technique commun (SQL → Python → ML).

3. Données et sources

Une exigence forte de ce projet était de n'utiliser **aucune donnée synthétique**. Les deux datasets retenus sont réels, publics, et largement reconnus dans la communauté data science comme référence pour leurs cas d'usage respectifs.

Volet	Dataset	Source	Volume	Cible
A — Credit Risk	Home Credit Default Risk	Kaggle (compétition)	307 511 clients, 8 tables liées	TARGET (0/1), 8.07% de défauts
B — Anomaly Detection	Credit Card Fraud Detection	Kaggle (ULB — Université Libre de Bruxelles)	284 807 transactions	Class (0/1), 0.173% de fraude

Home Credit Default Risk

Ce dataset provient d'une société de crédit à la consommation et regroupe la demande de crédit elle-même (*application_train*) ainsi que 7 tables secondaires retraçant l'historique du client : crédits déclarés auprès d'un bureau externe (*bureau*, *bureau_balance*), demandes de crédit précédentes auprès du même établissement (*previous_application*), et comportements de remboursement (*installments_payments*, *credit_card_balance*, *POS_CASH_balance*). Cette richesse relationnelle est précisément ce qui justifie l'usage de SQL pour l'agrégation, en amont du feature engineering Python.

Credit Card Fraud Detection

Ce dataset, largement utilisé comme référence académique en détection d'anomalies, contient des transactions par carte bancaire européenne sur 2 jours. Pour des raisons de confidentialité, la plupart des variables (V1 à V28) sont déjà des composantes issues d'une analyse en composantes principales (PCA) appliquée par les auteurs du dataset ; seules *Amount* (montant) et *Time* (secondes écoulées depuis la première transaction) restent interprétables directement.

Accès aux données

Les deux jeux de données ont été téléchargés via l'API officielle Kaggle. Le dataset Home Credit étant une ancienne compétition, son téléchargement a nécessité l'acceptation préalable du règlement sur la plateforme (contrainte d'accès classique des jeux de données de compétition, distincte des "datasets" Kaggle standards).

4. Architecture du pipeline

Le projet suit une chaîne de traitement linéaire, conforme à la spécification initiale *SQL → Python → ML → Imbalanced Learning → XGBoost → SHAP → Anomaly Detection* :

- **Ingestion** : chargement des CSV bruts (plus de 60 millions de lignes cumulées sur l'ensemble des tables) dans une base SQLite locale, avec indexation sur les clés de jointure (*SK_ID_CURR*, *SK_ID_BUREAU*).
- **Exploration SQL** : requêtes d'agrégation et de segmentation directement en base, pour valider les hypothèses avant tout chargement en mémoire Python.
- **Feature engineering** : construction d'une table de features unique par agrégation SQL des tables secondaires, jointes en Python à la table principale, puis enrichies de ratios métier et d'un encodage des variables catégorielles.
- **Modélisation** : comparaison de stratégies de gestion du déséquilibre par validation croisée, puis entraînement du modèle XGBoost final.
- **Évaluation** : métriques sur un test set isolé (jamais vu pendant l'entraînement), analyse du compromis precision/recall selon le seuil de décision.
- **Interprétabilité** : valeurs SHAP pour expliquer le modèle globalement et individuellement.
- **Détection d'anomalies** : Isolation Forest et DBSCAN appliqués aux transactions, avec visualisation comparative.

Choix technique : SQLite comme entrepôt intermédiaire

SQLite a été retenu plutôt qu'un simple chargement pandas direct des CSV pour deux raisons : (1) reproduire un environnement réaliste où les données résident dans un entrepôt interrogeable en SQL plutôt que dans des fichiers plats, conformément à la consigne du projet, et (2) permettre des agrégations (*GROUP BY*, jointures) efficaces sur des tables de plusieurs millions de lignes sans les charger intégralement en mémoire avant agrégation.

5. Volet A — Credit Risk (classification supervisée)

Cette section couvre l'ensemble du pipeline de scoring de risque de crédit, de l'exploration SQL initiale jusqu'à l'explication individuelle des décisions du modèle.

5.1 Exploration SQL

Avant toute modélisation, 5 requêtes SQL ont permis de valider les hypothèses structurantes du projet directement en base de données.

Distribution de la cible

TARGET	Nombre de clients	Pourcentage
0 (remboursé)	282 686	91.93%
1 (défaut)	24 825	8.07%

Lecture technique : seul 1 client sur 12 environ fait défaut. **Lecture métier** : ce déséquilibre est intrinsèque à l'activité de crédit — un établissement qui accepterait un taux de défaut proche de 50% ferait faillite. C'est précisément ce déséquilibre qui impose, plus loin dans le pipeline, un traitement spécifique (section 5.4) : un modèle naïf obtiendrait 92% d'accuracy en prédisant systématiquement "pas de défaut", ce qui serait inutile en pratique.

Taux de défaut par segment

La requête `02_default_rate_by_segment.sql` calcule le taux de défaut par genre, statut familial, type de revenu et niveau d'éducation. Résultats principaux :

Segment	Valeur	Taux de défaut
Genre	Homme	10.14%
Genre	Femme	7.00%
Éducation	Études secondaires incomplètes	10.93%
Éducation	Études supérieures	5.36%
Type de revenu	Sans emploi / congé maternité	36–40%*
Type de revenu	Retraité	5.39%

** Échantillons très réduits (5 à 22 clients) — taux à interpréter avec prudence, non représentatifs à grande échelle.*

Lecture métier : le gradient de risque est cohérent avec l'intuition économique (stabilité de l'emploi et niveau d'éducation corrélés à un risque plus faible). Ces résultats confirment, avant même toute modélisation, la pertinence de ces variables comme features prédictives.

Agrégations bureau externe et historique de demandes

Deux requêtes complémentaires (03 et 04) ont exploré le potentiel prédictif des tables secondaires : certains clients cumulent plus de 100 crédits déclarés au bureau externe, et le taux de refus sur les demandes de crédit précédentes varie de 0% à plus de 95% selon les clients. Cette hétérogénéité confirme l'intérêt d'agréger ces tables en features (section 5.3) plutôt que de les ignorer.

5.2 Analyse exploratoire (EDA)

L'EDA a été menée dans un notebook dédié (*01_eda_credit_risk.ipynb*), exécuté intégralement avec sorties conservées pour traçabilité. Six figures documentent les constats principaux.

Figure 1 — Distribution de la cible (TARGET)

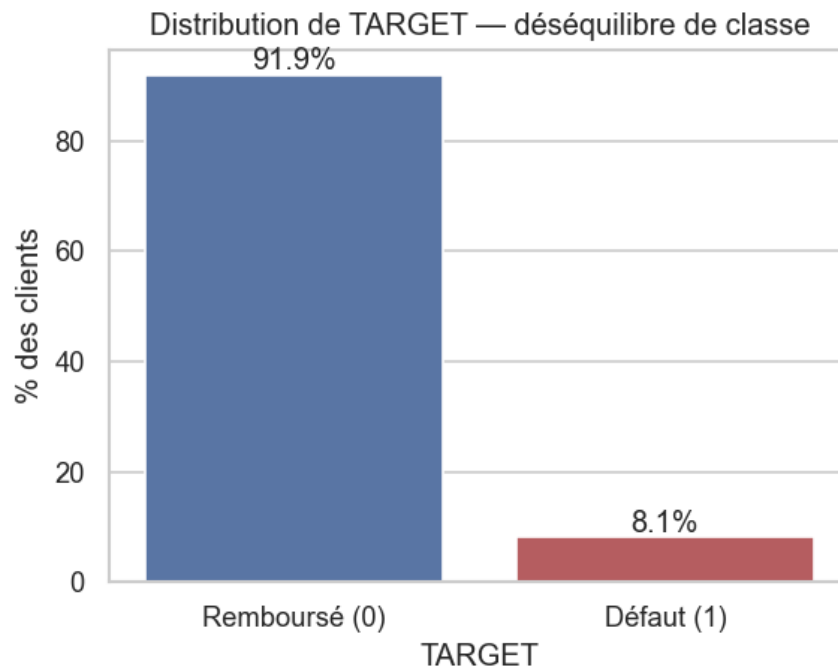
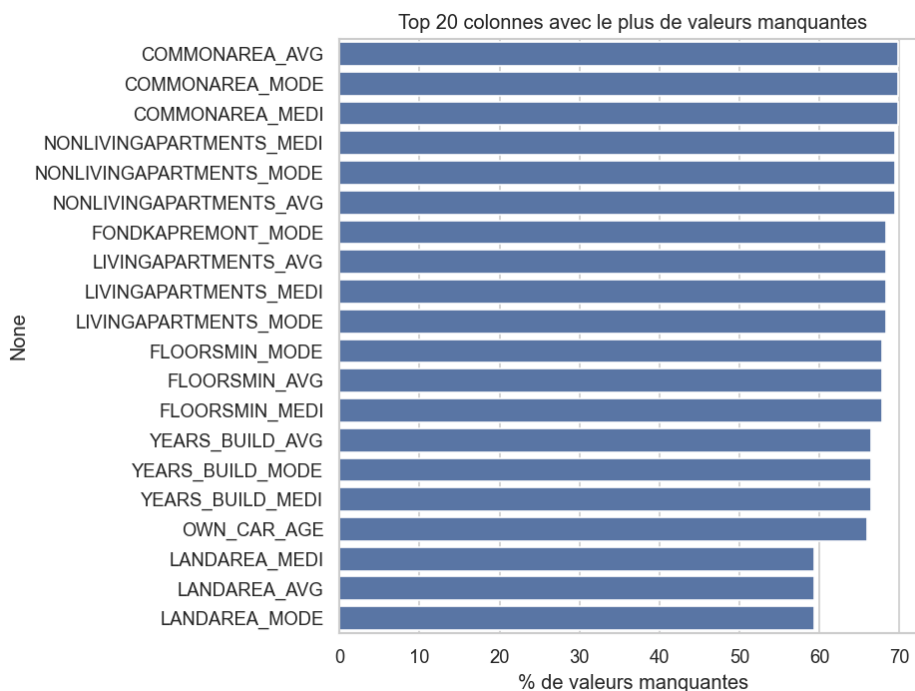


Diagramme en barres du pourcentage de clients par classe (0 = remboursé, 1 = défaut).

Explication technique : confirme visuellement le déséquilibre 92/8 déjà quantifié en SQL. **Explication métier** : ce graphique est le point de départ obligé de toute discussion sur le choix des métriques d'évaluation (section 5.6) — l'accuracy seule serait trompeuse, d'où le recours à Precision/Recall/F1/ROC-AUC.

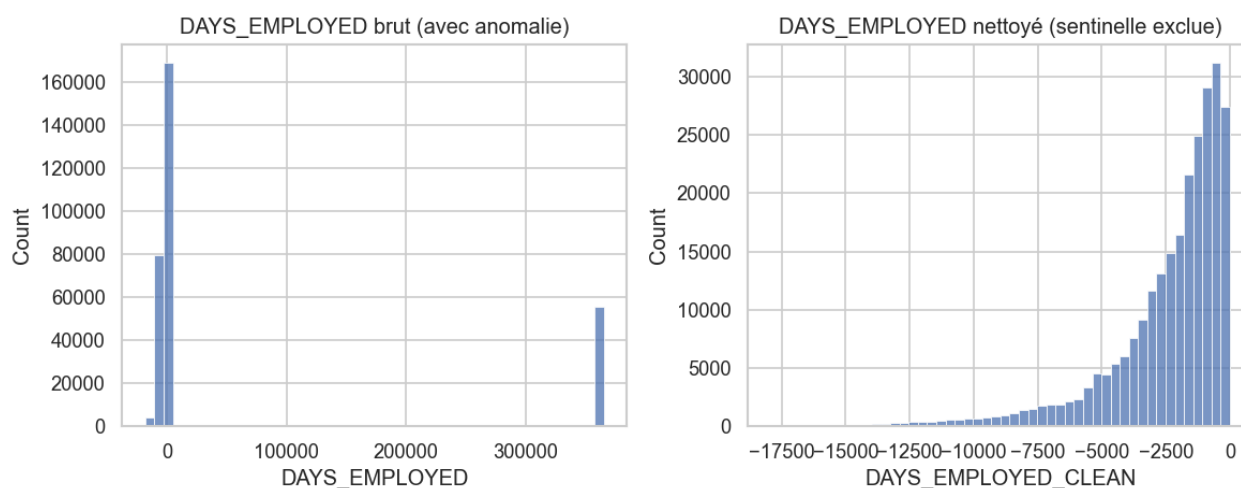
Figure 2 — Top 20 des colonnes avec le plus de valeurs manquantes



Barres horizontales triées par pourcentage de valeurs manquantes décroissant.

Explication technique : les colonnes les plus touchées sont des statistiques sur le logement du client (suffixes `_AVG`, `_MEDI`, `_MODE`), atteignant parfois plus de 50% de valeurs manquantes. **Décision méthodologique** : plutôt que d'imputer arbitrairement ou de supprimer ces colonnes, le choix a été fait de les conserver et de laisser XGBoost gérer nativement les valeurs manquantes (le framework apprend un chemin de décision par défaut pour les valeurs absentes à chaque nœud de split) — une valeur manquante peut elle-même être informative (ex. absence de déclaration = profil différent).

Figure 3 — Anomalie `DAYS_EMPLOYED` avant/après nettoyage

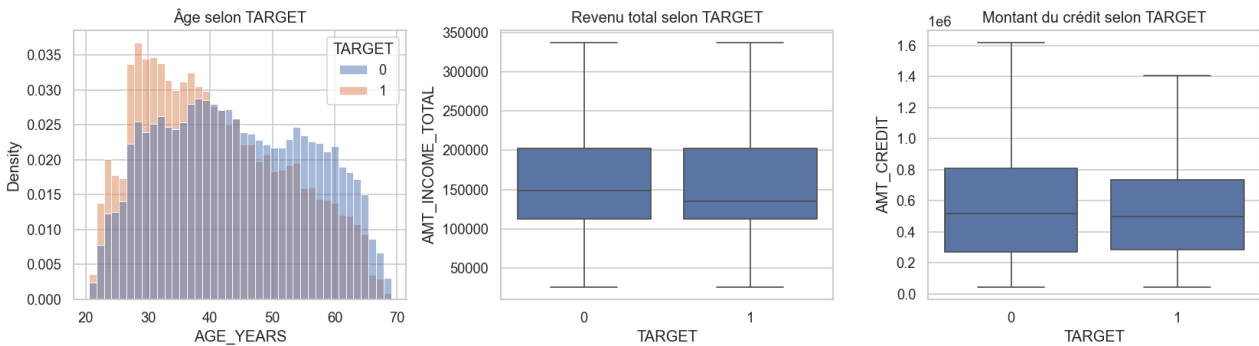


Histogrammes comparant la distribution brute (avec valeur sentinelle) et nettoyée.

Explication technique : le dataset contient une valeur sentinelle de 365 243 jours (environ 1000 ans) utilisée par le fournisseur de données pour coder l'absence d'emploi (retraités, etc.), ce qui pollue complètement la distribution brute si elle n'est pas traitée. **Traitement appliqué** : cette valeur a été extraite en un indicateur binaire dédié (`DAYS_EMPLOYED_ANOM`) puis remplacée par une valeur manquante dans la colonne numérique — préservant ainsi à la fois le signal "non-employé" (utile) et une

distribution numérique exploitable pour le reste des clients.

Figure 4 — Âge, revenu et montant du crédit selon TARGET

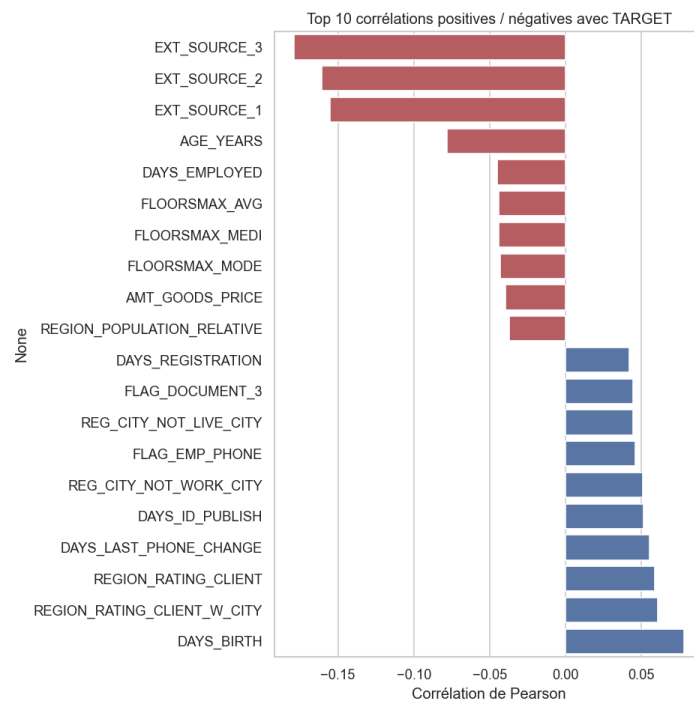


De gauche à droite : distribution de l'âge, boxplot du revenu, boxplot du montant du crédit, chacun scindé par TARGET.

Explication technique : les clients en défaut sont en moyenne plus jeunes ; les distributions de revenu et de montant de crédit se recouvrent largement entre les deux classes, signe qu'aucune variable seule n'est suffisante pour discriminer le risque (d'où la nécessité d'un modèle combinant de nombreuses features).

Explication métier : le lien âge/risque est cohérent avec la littérature du credit scoring (stabilité financière croissante avec l'âge / l'ancienneté professionnelle).

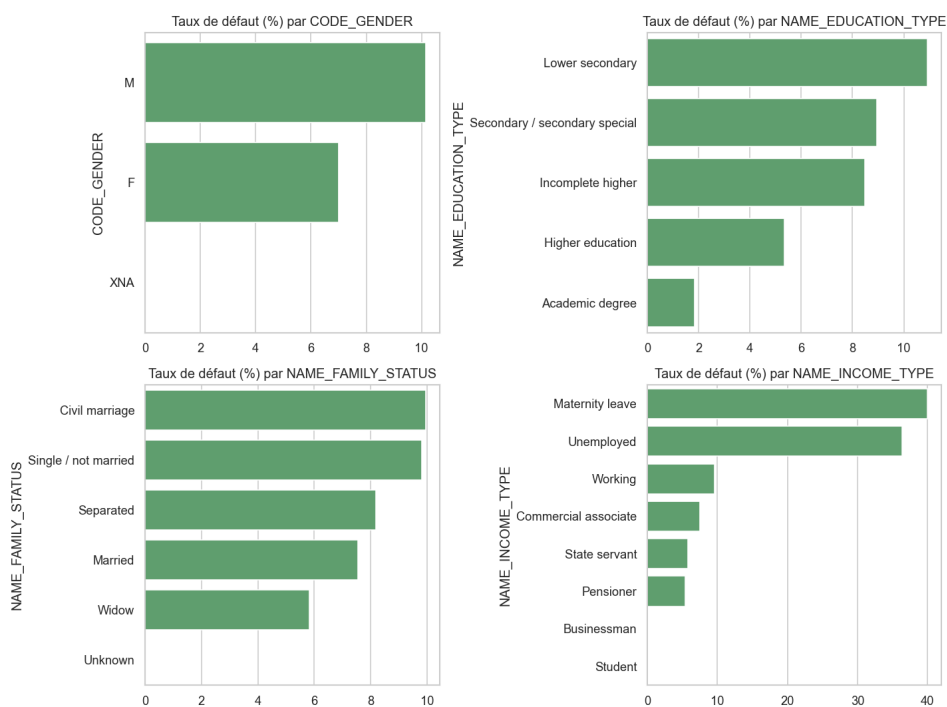
Figure 5 — Top 10 corrélations positives et négatives avec TARGET



Corrélation de Pearson entre chaque variable numérique et TARGET, triée.

Explication technique : cette figure est purement indicative — la corrélation linéaire ne capture pas les interactions non linéaires qu'XGBoost saura exploiter — mais elle permet un premier tri de santé avant modélisation, et confirme l'importance a priori des variables *EXT_SOURCE_** (scores de risque externes déjà fournis dans les données), qui seront confirmées comme dominantes par SHAP (section 5.7).

Figure 6 — Taux de défaut par segment catégoriel



Quatre sous-graphiques : genre, niveau d'éducation, statut familial, type de revenu.

Explication métier : reprise visuelle des résultats SQL de la section 5.1, utile pour une présentation à un public non technique (comité de crédit, conformité) — chaque barre représente un levier de segmentation potentiellement actionnable dans une politique de crédit, sous réserve de conformité réglementaire (le genre, notamment, est une variable sensible dont l'usage direct dans un score de crédit est encadré selon les juridictions).

5.3 Feature engineering

La table de features finale combine la demande de crédit elle-même (122 colonnes brutes) avec des agrégations des 5 tables secondaires du dataset, produisant **224 colonnes** pour **307 511 clients**.

Agrégations SQL par table secondaire

Table source	Niveau d'agrégation	Exemples de features créées
bureau	1 ligne / client	Nombre de crédits externes actifs/clôturés, dette totale cumulée, retard maximum observé
bureau_balance	1 ligne / client (via bureau)	Nombre moyen de mois suivis, total de mois en incident de paiement
previous_application	1 ligne / client	Nombre de demandes précédentes, taux de refus historique, ratio crédit accordé / demandé
installments_payments	1 ligne / client	Taux de retard de paiement, écart moyen entre montant dû et montant payé
credit_card_balance	1 ligne / client	Taux d'utilisation moyen de la limite de carte, taux de mois en dépassement (DPD)
POS_CASH_balance	1 ligne / client	Nombre moyen de mensualités restantes, taux de mois en retard

Features ratio métier

Six ratios ont été construits manuellement, reflétant des heuristiques classiques du credit scoring : *CREDIT_INCOME_RATIO* (montant du crédit / revenu), *ANNUITY_INCOME_RATIO* (mensualité / revenu — proche du "taux d'effort" réglementaire), *CREDIT_ANNUITY_RATIO* (durée implicite du crédit), *DAYS_EMPLOYED_PERC* (proportion de la vie passée en emploi), ainsi que *AGE_YEARS* et *EMPLOYED_YEARS* convertis en unités interprétables.

Encodage des variables catégorielles

Une règle générique a été appliquée : les variables catégorielles à faible cardinalité (≤ 15 modalités, ex. *NAME_EDUCATION_TYPE*) sont encodées en *one-hot*, tandis que les variables à forte cardinalité (ex. *ORGANIZATION_TYPE*, plus de 50 modalités) sont converties en fréquence d'occurrence — une approche qui évite l'explosion dimensionnelle du one-hot tout en conservant un signal exploitable.

Traitement des valeurs manquantes issues des agrégations

Une distinction a été faite entre deux types de valeurs manquantes après jointure : les colonnes de comptage (*nb_credits*, *nb_applications*...) sont remplies à 0 lorsqu'un client n'apparaît dans aucune table secondaire correspondante (absence d'historique = 0 occurrence, une hypothèse sûre) ; les colonnes de taux ou de moyenne (*refusal_rate*, *avg_balance*...) sont laissées à NaN plutôt que remplies par 0, car un taux de 0% de refus signifierait "aucun refus observé" alors que l'absence de valeur signifie "aucune donnée disponible" — deux situations sémantiquement différentes qu'un remplissage naïf aurait confondues.

Résultat final : 307 511 lignes × 224 colonnes, taux de valeurs manquantes moyen de 15.4%, sans doublon de clé primaire, prêt pour la modélisation.

5.4 Gestion du déséquilibre de classes

Trois stratégies ont été comparées par validation croisée stratifiée à 5 plis, sur l'ensemble d'entraînement (246 008 clients, 80% du total) :

- **Baseline** : XGBoost sans aucun traitement du déséquilibre.
- **scale_pos_weight** : pondération native XGBoost, calculée comme le ratio du nombre de négatifs sur le nombre de positifs dans le train set (ici 11.39), qui pénalise davantage les erreurs sur la classe minoritaire pendant l'entraînement.
- **SMOTE** (Synthetic Minority Over-sampling Technique) : génération d'exemples synthétiques de la classe minoritaire par interpolation entre plus proches voisins, appliquée uniquement sur les plis d'entraînement de la validation croisée (pour éviter toute fuite d'information vers les plis de validation).

Stratégie	ROC-AUC	PR-AUC	F1	Recall	Precision
Baseline	0.780	0.268	0.066	0.035	0.561
scale_pos_weight	0.778	0.267	0.291	0.673	0.185
SMOTE	0.772	0.257	0.039	0.020	0.565

Pourquoi scale_pos_weight a été retenu comme meilleur modèle

Ce choix mérite une explication détaillée car il n'est **pas** le résultat d'un critère unique appliqué mécaniquement. Le ROC-AUC et le PR-AUC — les deux métriques indépendantes du seuil de décision — sont quasiment identiques entre les trois stratégies (écarts de l'ordre de 0.01), ce qui pourrait laisser penser que n'importe laquelle convient. C'est un piège classique : ces métriques mesurent la qualité du *classement* produit par le modèle, pas son utilité au seuil opérationnel réellement utilisé.

En observant le F1-score et le recall **au seuil de décision par défaut (0.5)**, l'écart devient flagrant : le modèle *baseline*, sans pondération, ne détecte que 3.5% des défauts réels (recall = 0.035) — il prédit presque toujours "pas de défaut", ce qui est inutilisable en pratique malgré un PR-AUC apparemment correct. *scale_pos_weight* porte ce recall à 67.3%, au prix d'une précision plus faible (18.5% au lieu de 56.1%) — un compromis attendu et, dans un contexte de risque crédit, largement préférable : **manquer un défaut coûte généralement plus cher à l'établissement qu'examiner un dossier sain avec un peu plus d'attention.**

SMOTE, de son côté, obtient les moins bons résultats sur toutes les métriques. Ce résultat, bien que contre-intuitif au regard de sa popularité, s'explique par la haute dimensionnalité du jeu de features (222 colonnes) : SMOTE interpole entre plus proches voisins dans cet espace à haute dimension, où la notion de "proximité" perd en grande partie son sens géométrique (phénomène connu sous le nom de *curse of dimensionality*) — les exemples synthétiques générés sont donc moins représentatifs de véritables profils à risque, ce qui dégrade la capacité du modèle plutôt que de l'améliorer.

Enseignement méthodologique : ce cas illustre pourquoi il ne faut jamais sélectionner un modèle sur la seule base d'une métrique agrégée indépendante du seuil — l'analyse du comportement à un seuil opérationnel réaliste (ici via le F1 et le recall) est indispensable pour un cas d'usage métier concret.

5.5 Entraînement XGBoost

Le modèle final est un *XGBClassifier* entraîné avec les hyperparamètres suivants, choisis pour un compromis raisonnable entre capacité du modèle et risque de sur-apprentissage sur un dataset de cette taille :

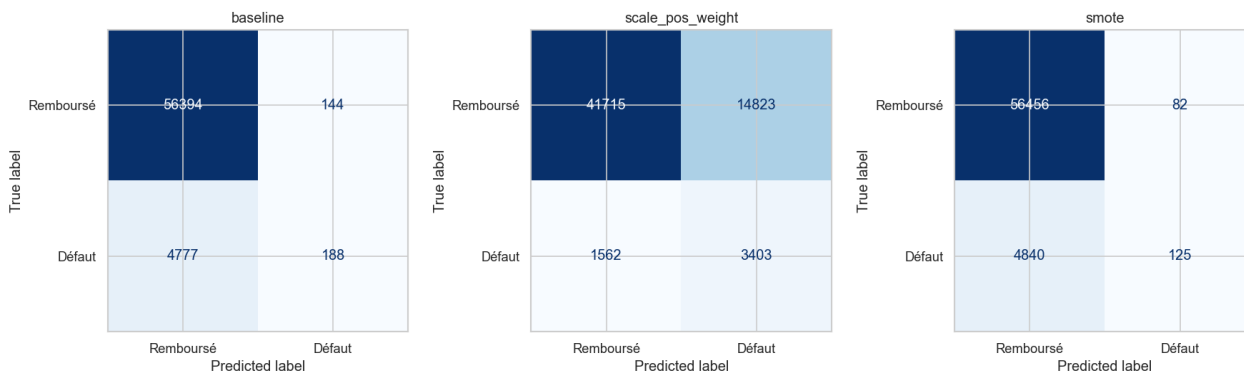
Hyperparamètre	Valeur	Rôle
n_estimators	300	Nombre d'arbres de boosting
max_depth	5	Profondeur maximale par arbre (limite la complexité individuelle de chaque arbre)
learning_rate	0.05	Taux d'apprentissage — valeur modérée compensée par un nombre d'arbres plus élevé, pour une convergence plus stable
subsample	0.8	Fraction des lignes utilisées par arbre (régularisation)
colsample_bytree	0.8	Fraction des colonnes utilisées par arbre (régularisation)
scale_pos_weight	11.39	Ratio négatifs/positifs du train set

XGBoost a été retenu comme algorithme principal (plutôt que, par exemple, une régression logistique ou une forêt aléatoire classique) pour trois raisons : sa gestion native des valeurs manquantes (pertinente ici vu les 15.4% de taux de manquants), sa robustesse reconnue sur données tabulaires hétérogènes, et sa compatibilité directe avec SHAP *TreeExplainer*, qui permet un calcul exact et rapide des valeurs SHAP (contrairement aux approximations nécessaires pour des modèles non basés sur des arbres).

5.6 Évaluation détaillée sur le test set

Le modèle retenu a été évalué sur les 61 503 clients du test set, mis de côté avant tout entraînement et jamais utilisés ni pour l'ajustement des hyperparamètres ni pour la validation croisée — garantissant une estimation non biaisée de la performance en conditions réelles.

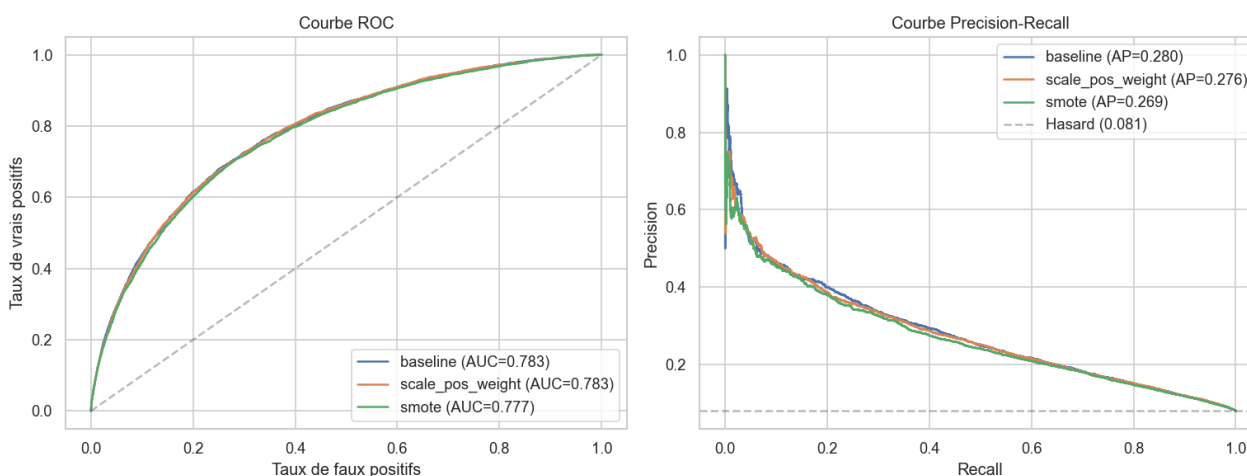
Figure 12 — Matrices de confusion des 3 stratégies (seuil = 0.5)



De gauche à droite : *baseline*, *scale_pos_weight*, *SMOTE*. Lignes = réel, colonnes = prédit.

Explication technique : la matrice *baseline* montre un nombre très faible de vrais positifs (défauts correctement identifiés) malgré un grand nombre de vrais négatifs — cohérent avec le recall de 3.5% observé en validation croisée. La matrice *scale_pos_weight* montre un meilleur équilibre, avec davantage de défauts détectés au prix de plus de faux positifs. **Explication métier** : chaque faux négatif de la matrice *baseline* représente un crédit à risque accordé sans alerte — le coût réel que le modèle pondéré cherche à réduire.

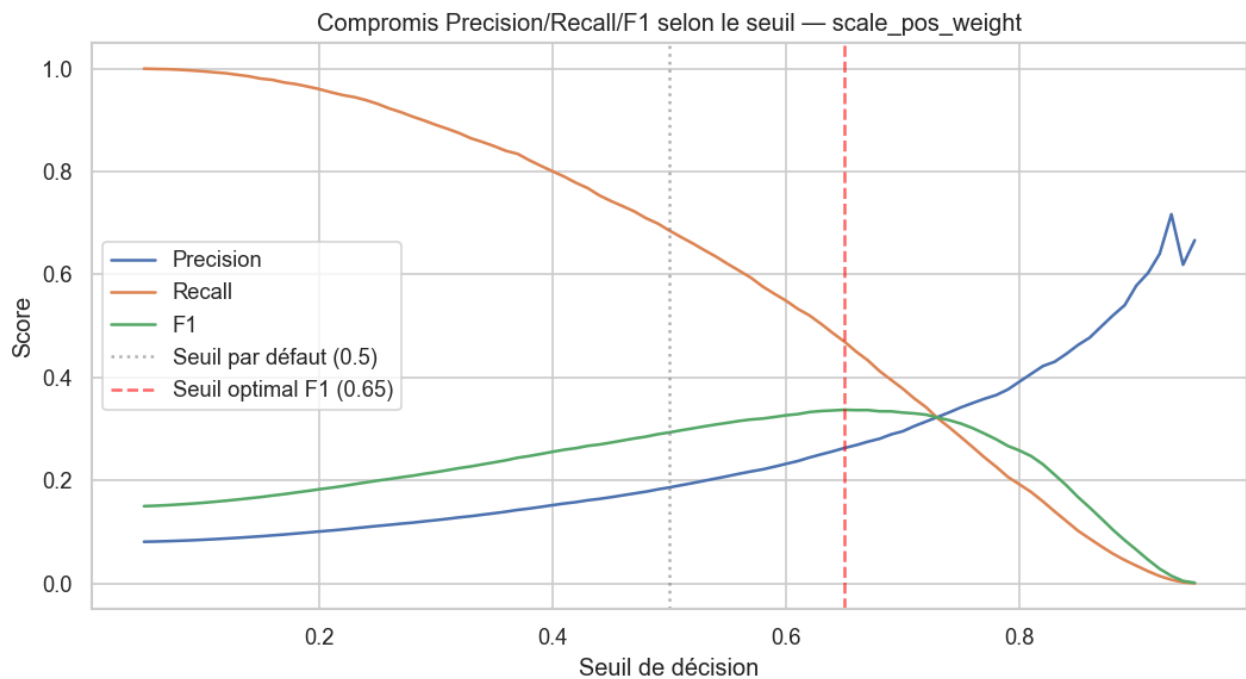
Figure 13 — Courbes ROC et Precision-Recall comparées



Gauche : courbe ROC (taux de vrais positifs vs taux de faux positifs). Droite : courbe Precision-Recall, avec la ligne du hasard en pointillés.

Explication technique : les trois courbes ROC sont quasiment superposées (AUC ~0.78 pour les trois), confirmant que les trois modèles classent les clients de façon comparable — c'est la courbe Precision-Recall, plus sensible au déséquilibre de classes, qui est la plus informative ici. La ligne du hasard (à 0.08, le taux de base de défaut) sert de référence : un modèle utile doit rester significativement au-dessus de cette ligne sur toute la plage de recall.

Figure 14 — Compromis Precision/Recall/F1 selon le seuil de décision



Évolution des trois métriques pour le modèle `scale_pos_weight`, seuil variant de 0.05 à 0.95. Ligne pointillée grise : seuil par défaut (0.5). Ligne rouge : seuil optimal au sens du F1 (0.65).

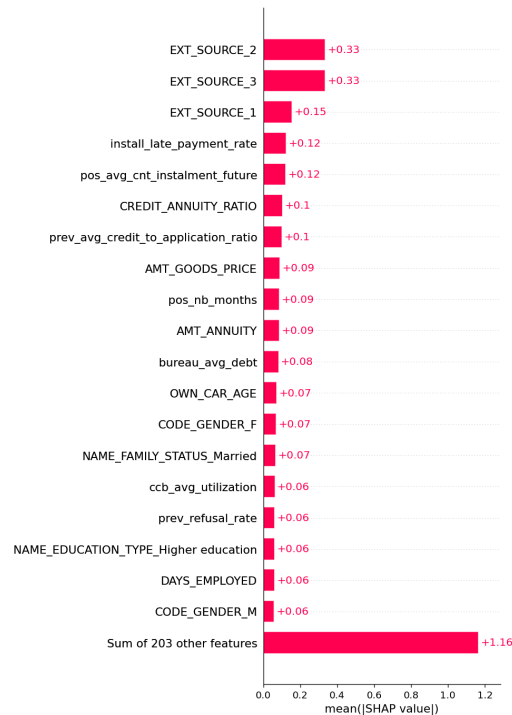
Explication technique : cette figure est la plus directement actionnable du rapport. Elle montre qu'au seuil par défaut de 0.5, le modèle privilégie fortement le recall (0.685) au détriment de la precision (0.187) ; en déplaçant le seuil à 0.65, le F1 passe de 0.293 à 0.337, avec un meilleur équilibre (precision 0.263, recall 0.470). **Explication métier** : le choix du seuil final ne devrait pas être purement statistique (maximiser le F1) mais refléter le **coût réel** estimé par l'établissement pour un faux négatif (perte sur défaut non anticipé) comparé à un faux positif (coût d'instruction manuelle ou de refus d'un bon client) — cette courbe fournit exactement les points de données nécessaires pour arbitrer ce choix avec les équipes métier une fois ces coûts connus.

Seuil	ROC-AUC	Precision	Recall	F1
0.50 (par défaut)	0.783	0.187	0.685	0.293
0.65 (optimal F1)	0.783	0.263	0.470	0.337

5.7 Interprétabilité (SHAP)

SHAP (SHapley Additive exPlanations) a été utilisé pour expliquer le modèle final, avec un *TreeExplainer* adapté à XGBoost, calculé sur un échantillon de 2000 clients du test set. SHAP répond à deux besoins distincts : comprendre le modèle **globalement** (quelles features comptent en moyenne) et expliquer une prédiction **individuelle** (pourquoi ce client précis obtient ce score).

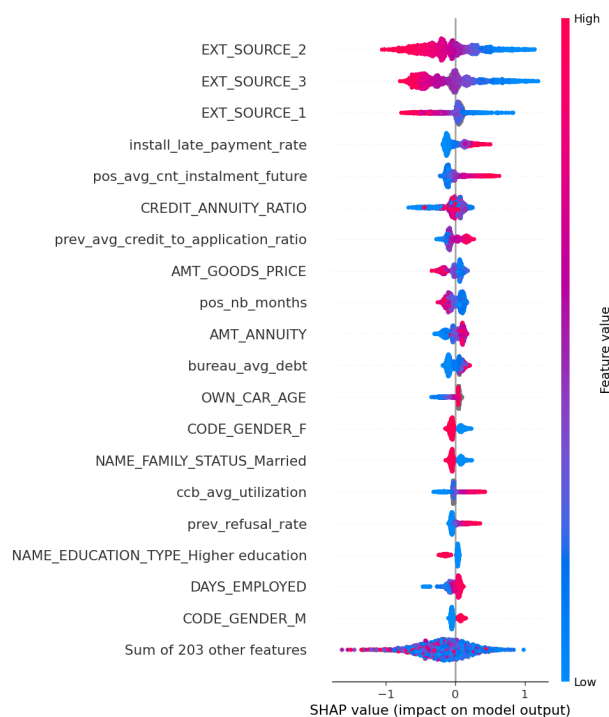
Figure 15 — Importance globale des features (valeur SHAP absolue moyenne)



Classement des 20 features les plus influentes par magnitude moyenne de leur contribution SHAP.

Explication technique : contrairement à l'importance de features native de XGBoost (basée sur la fréquence ou le gain des splits), l'importance SHAP mesure la contribution réelle de chaque variable à l'écart entre la prédiction et la valeur de base moyenne — une mesure plus fidèle à l'impact effectif sur la décision finale.

Figure 16 — Beeswarm plot : direction et magnitude de l'effet

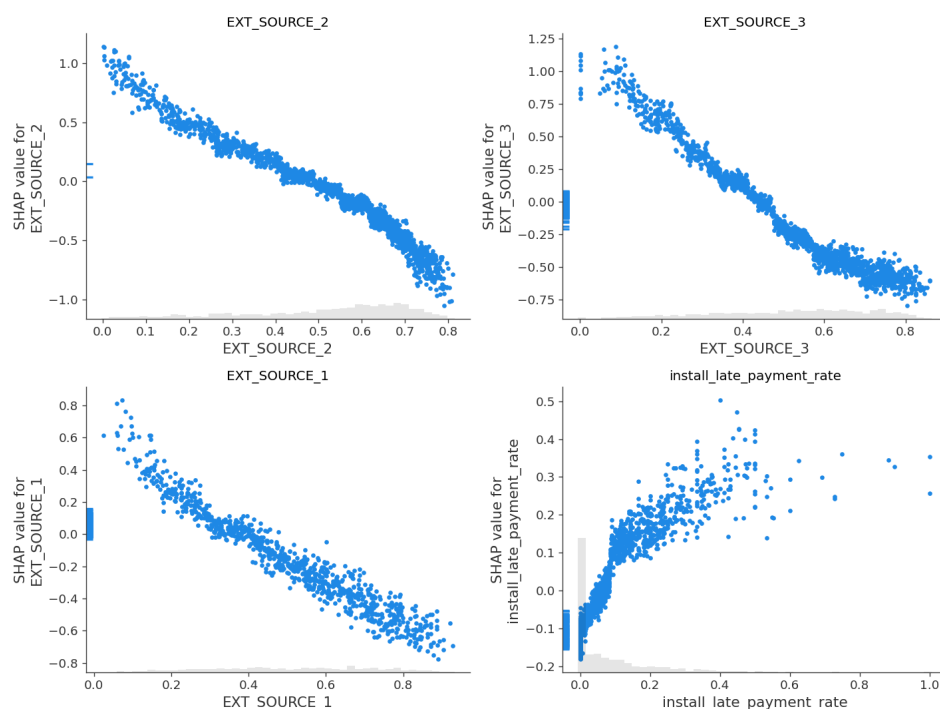


Chaque point est un client. Position horizontale = impact SHAP sur la prédiction (gauche = pousse vers "pas de défaut", droite = pousse vers "défaut"). Couleur = valeur de la feature (bleu = faible, rouge = élevée).

Explication technique : ce graphique révèle la *direction* de l'effet, information absente du bar plot précédent. On observe par exemple que des valeurs élevées (rouge) d'*EXT_SOURCE_2* et *EXT_SOURCE_3* (scores de crédit externes) sont associées à un effet négatif sur le risque (gauche du graphique) — cohérent avec leur rôle de scores de solvabilité, où une valeur élevée signale un bon profil. À l'inverse, un *install_late_payment_rate* (taux de retard de paiement historique) élevé pousse la prédiction vers le défaut. **Explication métier** : ce constat valide, avec des données réelles, l'intuition selon laquelle l'historique de paiement est un signal de risque de premier ordre — un principe déjà bien établi en scoring de crédit traditionnel, mais ici quantifié et confirmé statistiquement sur ce portefeuille précis.

Validation du feature engineering : le fait que plusieurs variables construites lors de l'agrégation SQL (*install_late_payment_rate*, *pos_avg_cnt_instalment_future*, *CREDIT_ANNUITY_RATIO*, *prev_avg_credit_to_application_ratio*, *bureau_avg_debt*, *ccb_avg_utilization*, *prev_refusal_rate*) apparaissent dans le top 20 — aux côtés des seules variables déjà présentes nativement dans les données (*EXT_SOURCE_**, *AMT_GOODS_PRICE*, *AMT_ANNUITY*) — constitue une validation directe de l'effort d'agrégation des tables secondaires réalisé en section 5.3 : ces features apportent un signal prédictif réel, non redondant avec les données brutes.

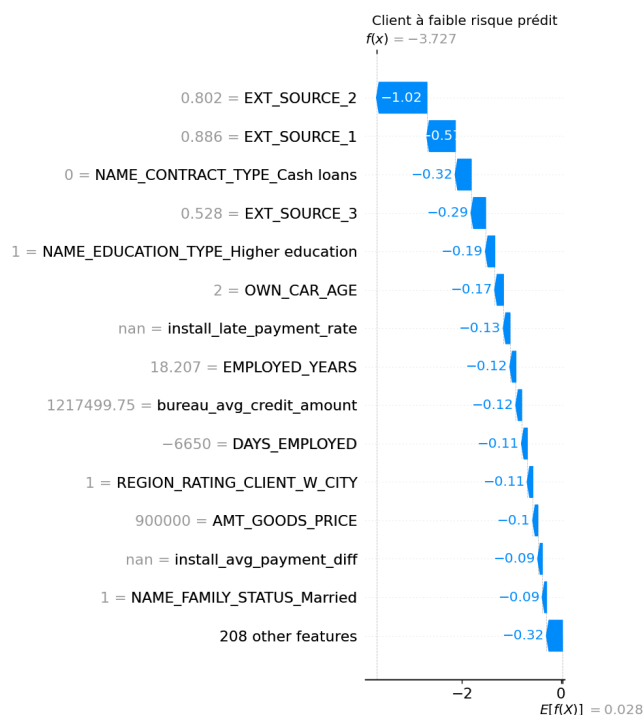
Figure 17 — Dependence plots sur les 4 features les plus influentes



Relation entre la valeur de chaque feature (axe horizontal) et son impact SHAP (axe vertical), pour les 4 features dominantes.

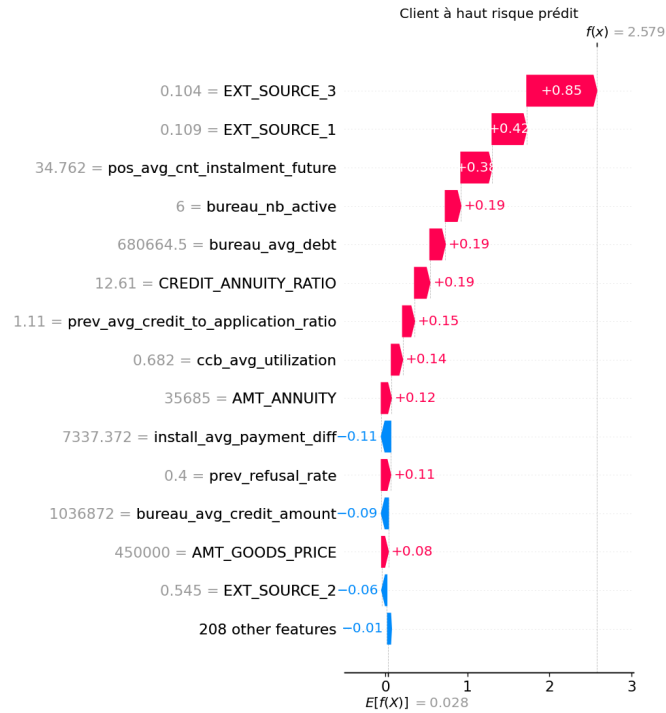
Explication technique : ces graphiques permettent de vérifier si la relation entre une feature et le risque est monotone (linéaire ou quasi-linéaire) ou plus complexe (seuils, effets non linéaires) — une information invisible dans le simple coefficient de corrélation de la figure 5. C'est un des apports spécifiques de SHAP par rapport à une analyse de corrélation classique : il capture les relations non linéaires qu'un modèle d'arbres comme XGBoost peut apprendre.

Figure 18 — Explication individuelle : client à faible risque prédit



Décomposition de la prédiction pour un client spécifique, du score de base (moyenne du portefeuille) jusqu'à la prédiction finale, feature par feature.

Figure 19 — Explication individuelle : client à haut risque prédit



Même lecture que la figure 18, pour un client au profil opposé.

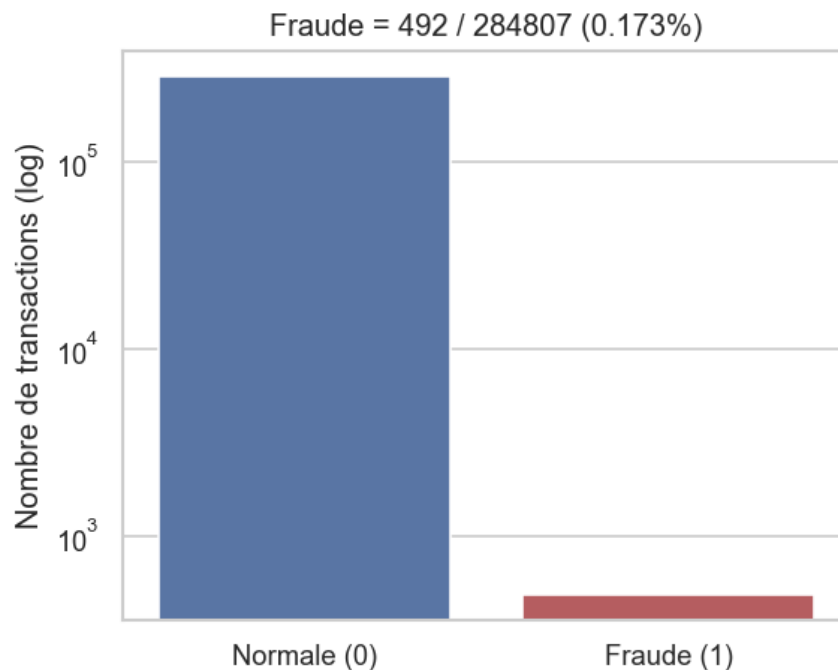
Explication technique : pour le client à haut risque (figure 19), les principales contributions positives (poussant vers le défaut) proviennent de scores externes faibles ($EXT_SOURCE_3 = 0.104$, contribution +0.85 en log-odds ; $EXT_SOURCE_1 = 0.109$, contribution +0.42) et d'un nombre élevé de mensualités restantes sur des crédits POS ($pos_avg_cnt_instalment_future = 34.8$, +0.38). **Explication métier** : ce type de graphique est directement exploitable pour justifier une décision de crédit auprès d'un client ou d'un régulateur — il répond concrètement à la question "pourquoi ce dossier a-t-il été refusé (ou accepté) ?" avec une décomposition quantifiée et vérifiable, contrairement à un modèle "boîte noire" qui ne fournirait qu'un score sans justification.

6. Volet B — Anomaly Detection (non supervisé)

Ce volet traite la détection de transactions frauduleuses comme un problème d'apprentissage non supervisé : les deux algorithmes utilisés (Isolation Forest, DBSCAN) n'ont **jamais accès au label Class pendant l'entraînement** — ce label ne sert qu'à évaluer a posteriori la qualité de détection, une distinction méthodologique essentielle à comprendre pour ce volet.

6.1 Analyse exploratoire (EDA)

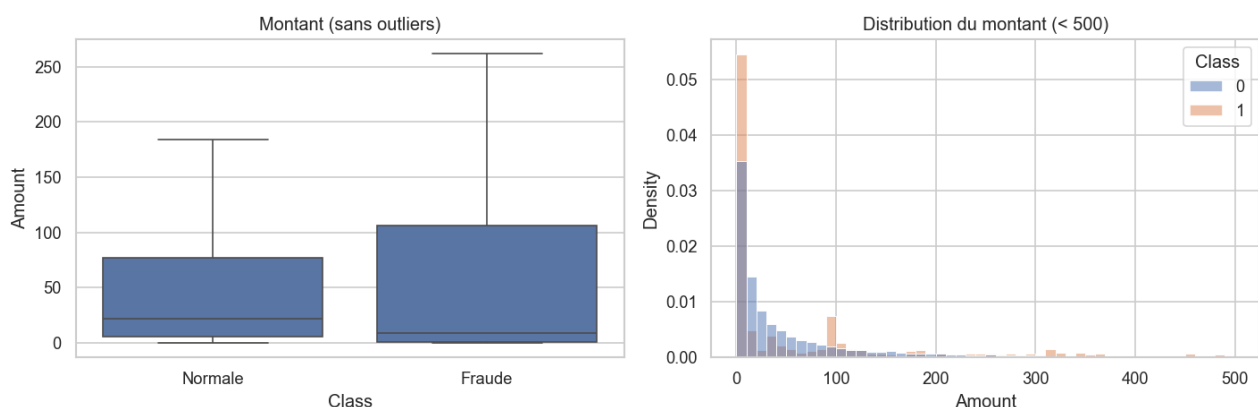
Figure 7 — Distribution des classes (échelle logarithmique)



Nombre de transactions normales vs frauduleuses, axe vertical en échelle log pour rendre visible la classe minoritaire.

Explication technique : seules 492 transactions sur 284 807 sont frauduleuses (0.173%) — un déséquilibre encore plus extrême que celui du volet A (8%). **Explication métier** : ce niveau de rareté justifie directement le choix d'approches non supervisées plutôt qu'une classification supervisée classique, qui disposerait de trop peu d'exemples positifs pour généraliser correctement, et qui en production devrait aussi détecter des *nouveaux* types de fraude jamais vus à l'entraînement — un scénario où le non supervisé a un avantage structurel.

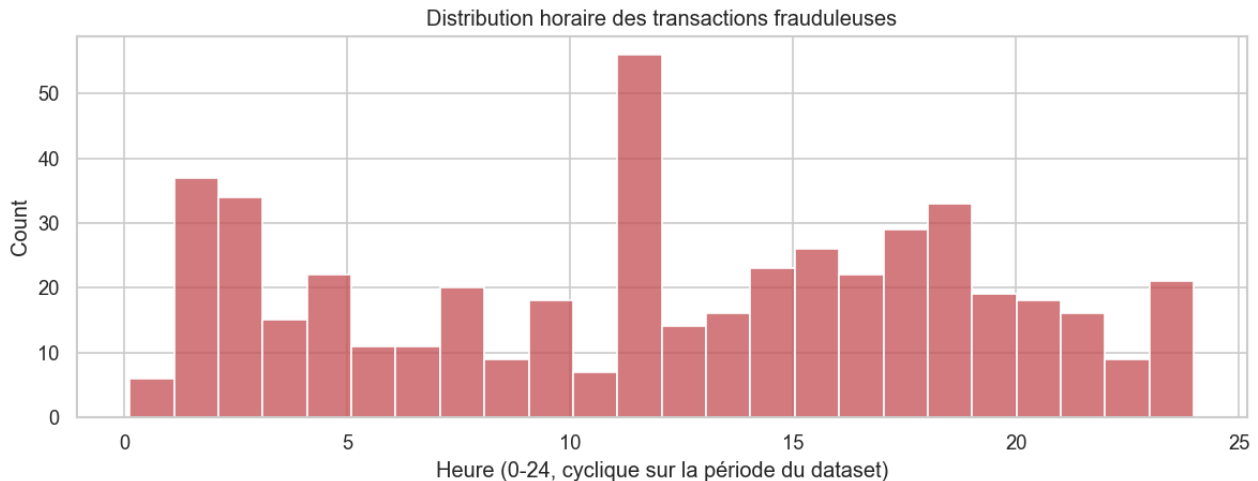
Figure 8 — Distribution du montant des transactions (normal vs fraude)



Boxplot (sans outliers) et histogramme (< 500) comparant les montants selon la classe.

Explication métier : les transactions frauduleuses ont un montant moyen légèrement supérieur (122€ contre 88€) mais avec une distribution différente — peu de très grosses fraudes, contrairement à une intuition naïve qui associerait fraude et montants extrêmes. Ce constat nuance une règle métier simpliste du type "bloquer les transactions au-dessus d'un certain montant", qui laisserait passer la majorité des fraudes réelles de ce dataset.

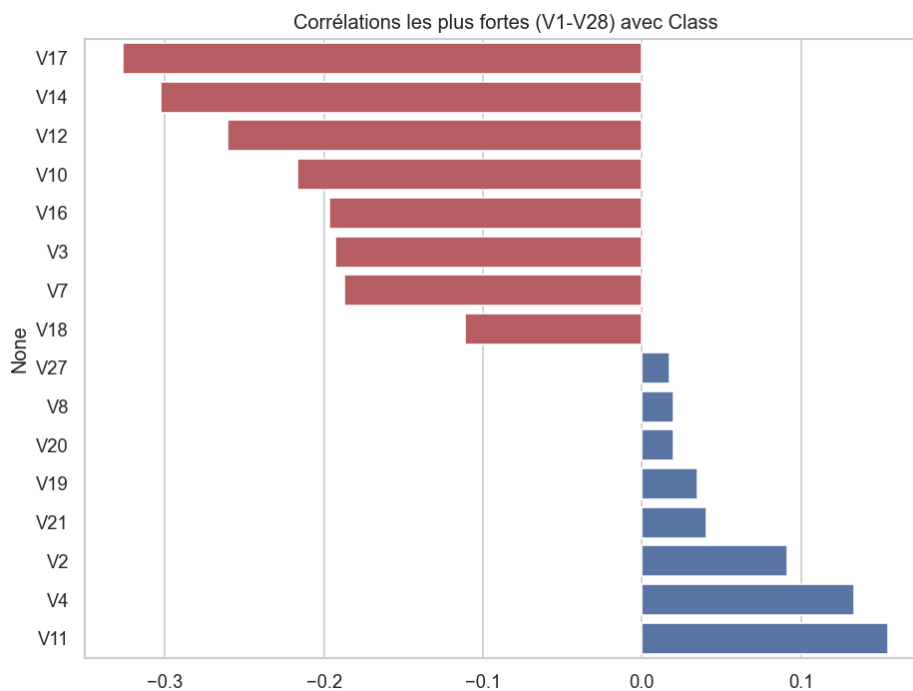
Figure 9 — Répartition horaire des transactions frauduleuses



Histogramme du nombre de fraudes par heure de la journée (cyclique sur la période couverte par le dataset).

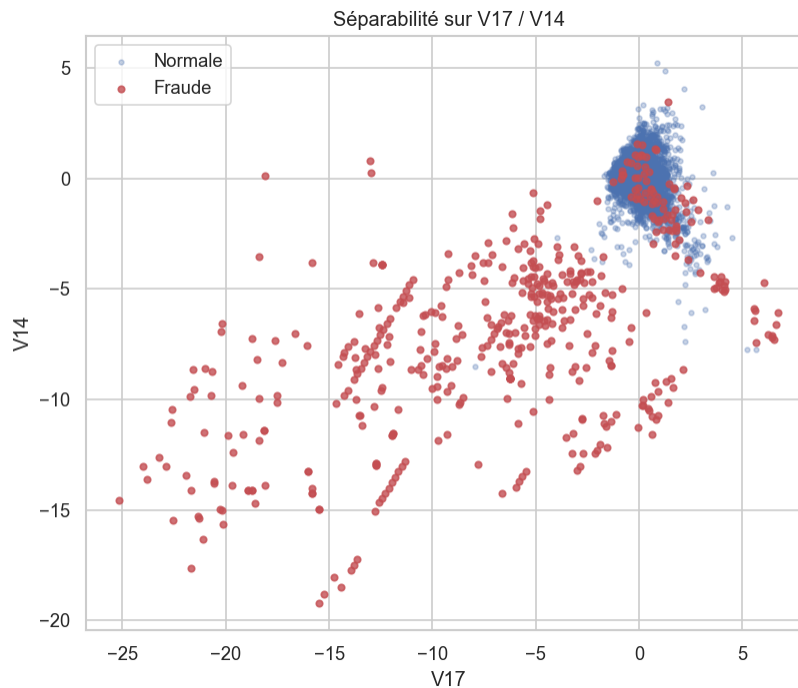
Explication métier : une distribution horaire non uniforme (si observée) pourrait justifier des règles de vigilance renforcée à certaines heures, en complément du scoring algorithmique — une piste d'amélioration opérationnelle simple à mettre en œuvre.

Figure 10 — Corrélations les plus fortes entre composantes PCA et Class



Top 8 corrélations positives et négatives parmi les variables V1-V28.

Figure 11 — Séparabilité visuelle sur les 2 composantes les plus corrélées (V17, V14)



Nuage de points : échantillon de transactions normales (bleu) et l'ensemble des fraudes (rouge), projetées sur les composantes V17 et V14.

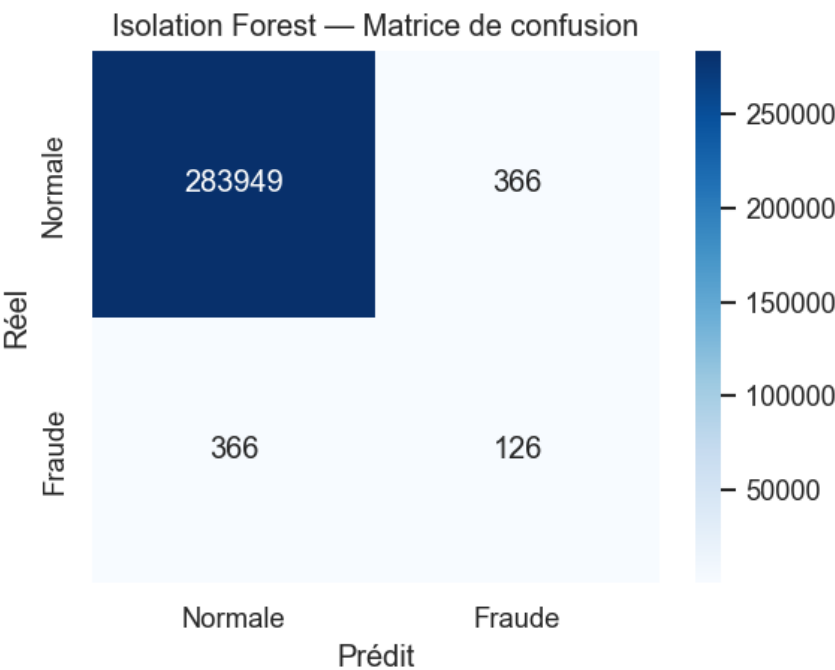
Explication technique : ce nuage de points est la validation visuelle la plus parlante du potentiel de détection non supervisée sur ce dataset — les fraudes (points rouges) se distinguent nettement du cœur de la distribution normale (points bleus), formant un nuage étalé en périphérie plutôt qu'un cluster compact confondu avec les transactions normales. C'est exactement le type de structure que les méthodes utilisées en sections 6.2 et 6.3 sont conçues pour détecter : l'isolement (Isolation Forest) et la faible densité locale (DBSCAN).

6.2 Isolation Forest

Isolation Forest a été entraîné sur l'**intégralité** des 284 807 transactions standardisées (moyenne 0, écart-type 1 sur chaque variable), avec le paramètre *contamination* fixé au taux de fraude réel observé (0.173%) comme prior raisonnable — un choix justifiable ici car le taux de fraude global est généralement connu ou estimable par l'établissement, même si les transactions individuellement frauduleuses ne le sont pas.

Le principe de l'algorithme consiste à isoler chaque observation par des partitionnements aléatoires successifs de l'espace des features : une observation atypique (donc une fraude potentielle) nécessite en moyenne moins de partitionnements pour être isolée qu'une observation normale noyée dans la masse des données. Ce principe, contrairement à DBSCAN, ne repose pas sur une notion explicite de distance, ce qui le rend nativement plus robuste aux données de haute dimension et capable de passer à l'échelle sans échantillonnage.

Figure 20 — Isolation Forest : matrice de confusion (données complètes)



Comparaison des 492 anomalies détectées aux 492 fraudes réelles, sur l'ensemble des 284 807 transactions.

Métrique	Valeur
Precision	0.256
Recall	0.256
F1-score	0.256
Average Precision (score continu)	0.164

Explication technique : precision, recall et F1 sont identiques ici car le paramètre *contamination* a été fixé exactement au taux de fraude réel — le nombre d'anomalies prédites (492) égale donc exactement le nombre de fraudes réelles, ce qui égalise mécaniquement precision et recall. Le score *Average Precision* (0.164), lui, ne dépend pas de ce choix de seuil et reflète la qualité de classement continu du modèle.

Explication métier : un quart des transactions signalées comme anormales sont effectivement

frauduleuses (precision 25.6%) — un taux nettement supérieur au taux de base de 0.17%, ce qui représente un gain d'efficacité considérable pour une équipe d'investigation manuelle qui devrait sinon examiner l'ensemble des transactions.

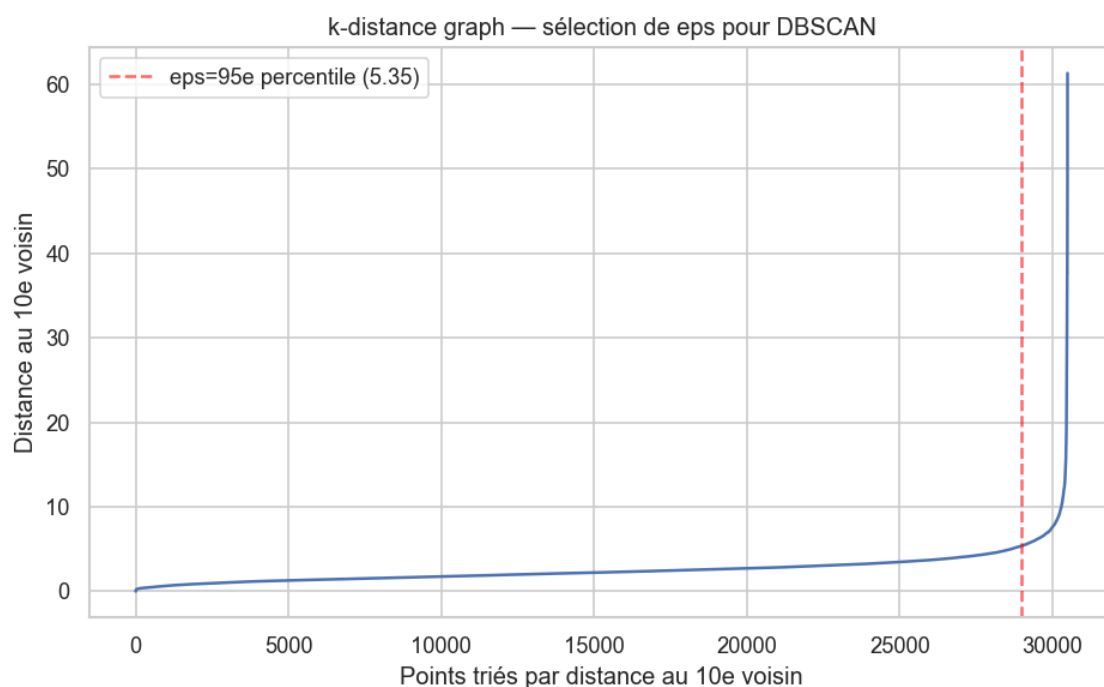
6.3 DBSCAN

DBSCAN pose un défi de passage à l'échelle : son coût de calcul croît de façon quadratique avec le nombre de points dans le pire cas, ce qui le rend impraticable sur 284 807 transactions × 30 dimensions dans le cadre de ce projet. Un **échantillon stratifié** de 30 492 transactions a donc été constitué (l'intégralité des 492 fraudes + 30 000 transactions normales tirées aléatoirement), une pratique standard pour démontrer une méthode coûteuse sur un sous-ensemble représentatif.

Un problème méthodologique rencontré et corrigé

La sélection du paramètre *eps* (rayon de voisinage) de DBSCAN illustre une difficulté réelle et instructive rencontrée pendant le projet, documentée ici pour sa valeur pédagogique.

Figure 21 — k-distance graph ayant guidé la sélection finale de eps



Distance de chaque point à son 10e plus proche voisin, triée par ordre croissant. La ligne rouge marque le eps finalement retenu (95e percentile).

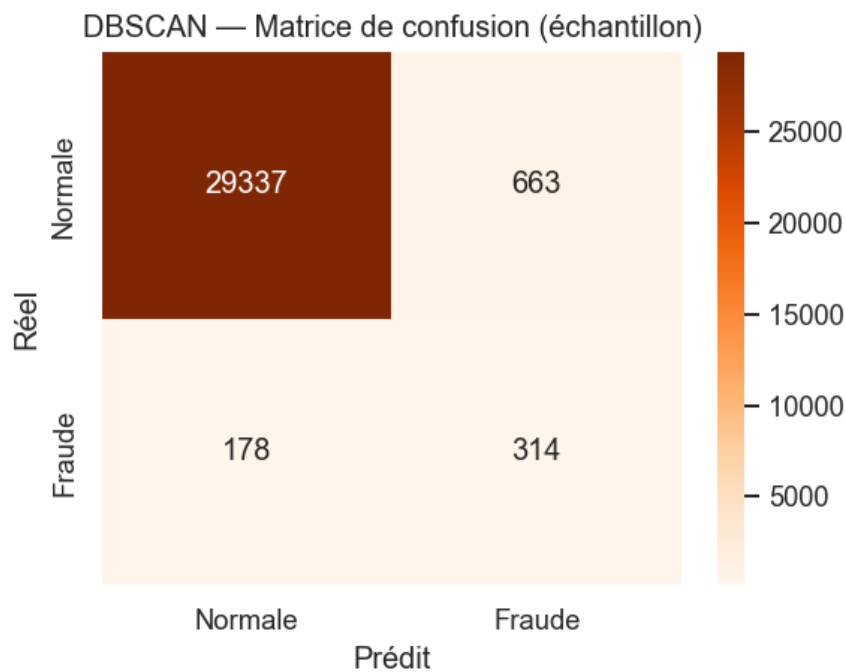
Première tentative (échec) : la méthode classique du "coude" (chercher le point de courbure maximale sur le k-distance graph) a été appliquée automatiquement et a sélectionné *eps* = 0.24, une valeur beaucoup trop faible. Résultat : 99.81% des points ont été classés comme "bruit" (donc anomalie), rendant la detection inutilisable.

Diagnostic : la courbe des k-distances présente une distribution à "queue lourde" — la grande majorité des points ont une distance au 10e voisin comprise entre 0.3 et 7, mais quelques transactions extrêmes (même après standardisation) font grimper la courbe jusqu'à plus de 60 sur le dernier centile. La méthode du coude, qui cherche le point le plus éloigné de la droite reliant les deux extrémités de la courbe, est mathématiquement dominée par cette queue extrême et détecte un "faux coude" très en amont de la courbe.

Deuxième tentative (échec partiel) : exclure les 2% de valeurs les plus extrêmes du calcul du coude a légèrement amélioré la situation ($eps = 0.73$) mais 93.62% des points restaient classés comme bruit — la queue lourde de cette distribution particulière nécessitait une troncature beaucoup plus importante pour que la méthode du coude fonctionne correctement.

Solution retenue : plutôt que de continuer à ajuster manuellement le seuil de troncature, une heuristique alternative plus robuste a été adoptée — fixer eps au **95e percentile** des distances au k-ième voisin ($eps = 5.35$), une approche standard en pratique lorsque la méthode du coude échoue sur des distributions non standard.

Figure 22 — DBSCAN : matrice de confusion (échantillon, après correction)



3 clusters trouvés, 977 points classés comme bruit sur l'échantillon de 30 492 transactions (3.20%).

Métrique	Valeur
Precision	0.321
Recall	0.638
F1-score	0.428

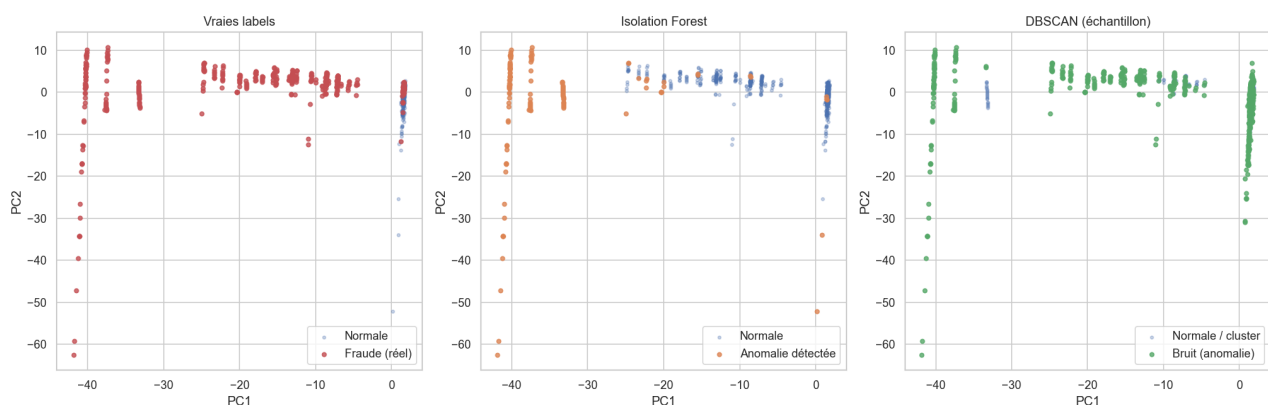
Explication métier : après correction, DBSCAN détecte 63.8% des fraudes présentes dans l'échantillon avec une precision de 32.1% — un résultat sensiblement meilleur qu'Isolation Forest sur cette même dimension de comparaison, mais obtenu au prix d'un échantillonnage qui ne serait pas praticable tel quel en production sur un flux de transactions continu et volumineux.

6.4 Comparaison et visualisation

Méthode	Données évaluées	Precision	Recall	F1
Isolation Forest	284 807 (totalité)	0.256	0.256	0.256
DBSCAN	30 492 (échantillon stratifié)	0.321	0.638	0.428

Arbitrage métier entre les deux approches : DBSCAN affiche de meilleures métriques sur cette comparaison, mais cette conclusion doit être nuancée par une considération opérationnelle majeure — Isolation Forest a été évalué sur l'intégralité du flux de données tandis que DBSCAN n'a pu être évalué que sur un échantillon représentant environ 10.7% du volume total, en raison de sa complexité de calcul. En production, où le volume de transactions à surveiller est typiquement continu et croissant, cette contrainte de passage à l'échelle favoriserait Isolation Forest malgré ses métriques plus modestes sur cette évaluation — sauf à accepter un DBSCAN appliqué par lots (*batch*) sur des fenêtres temporelles limitées plutôt qu'en flux continu.

Figure 23 — Comparaison visuelle des anomalies détectées (projection PCA 2D)



Trois panneaux projetés sur les 2 premières composantes principales de l'espace standardisé : vraies labels (gauche), anomalies Isolation Forest (centre), bruit DBSCAN (droite).

Explication technique : cette projection en 2 dimensions (à partir de 30 dimensions d'origine) permet une comparaison visuelle directe des trois vues. On observe que les deux méthodes identifient des anomalies dans des zones globalement cohérentes avec la position réelle des fraudes (panneau de gauche), avec des différences de couverture attendues compte tenu de leurs principes algorithmiques distincts (isolement par partitionnement aléatoire vs densité locale de voisinage).

7. Discussion, limites et recommandations

Limites du volet A (Credit Risk)

- Le modèle a été évalué sur un split temporel non contrôlé (split aléatoire stratifié) — en production, une validation par découpage temporel (entraînement sur une période, test sur la période suivante) serait plus représentative de la dérive du portefeuille dans le temps.
- Le seuil de décision optimal (0.65) a été choisi par maximisation du F1, une métrique statistique neutre — un déploiement réel nécessiterait un chiffrage explicite du coût métier d'un faux négatif vs faux positif pour arbitrer ce seuil plus précisément.
- Certaines variables utilisées (genre, statut familial) sont des données sensibles dont l'usage direct dans un score de crédit peut être encadré réglementairement selon les juridictions — un déploiement réel nécessiterait une revue de conformité (équité algorithmique, disparate impact) non menée dans le cadre de ce projet portfolio.

Limites du volet B (Anomaly Detection)

- DBSCAN a été évalué sur un échantillon, non sur le flux complet — sa performance en production sur l'intégralité du volume reste à valider, potentiellement via une architecture de traitement par lots.
- Les deux méthodes ont été évaluées a posteriori à l'aide de labels connus (*Class*), ce qui n'est qu'une approximation du cas d'usage réel non supervisé où ces labels ne seraient pas disponibles — l'exercice valide la pertinence des méthodes sur ce dataset précis, mais leur calibrage en production nécessiterait un processus de revue humaine des alertes générées.
- Le seuil *contamination* d'Isolation Forest a été fixé au taux de fraude historique observé, une hypothèse qui suppose que ce taux reste stable dans le temps — une hypothèse à surveiller en production (dérive du taux de fraude).

Recommandations pour une suite opérationnelle

- Volet A : chiffrer le coût métier réel des faux négatifs/positifs avec les équipes risque pour affiner le seuil de décision, et mener une revue de biais algorithmique avant tout déploiement en production.
- Volet B : envisager une architecture hybride combinant Isolation Forest en continu (pour sa scalabilité) et une analyse DBSCAN périodique par lots sur des fenêtres temporelles, pour capter des structures de densité locale que l'Isolation Forest ne modélise pas explicitement.
- Mettre en place une boucle de rétroaction (les alertes confirmées ou infirmées par les équipes d'investigation viendraient réentraîner les modèles) pour transformer progressivement le volet B en système semi-supervisé.

8. Annexes — stack technique et reproductibilité

Stack technique utilisée

Catégorie	Outils
Stockage / SQL	SQLite (chargement par lots, index sur clés de jointure)
Manipulation de données	pandas, numpy
Modélisation	XGBoost (XGBClassifier), scikit-learn
Gestion du déséquilibre	imbalanced-learn (SMOTE), scale_pos_weight natif XGBoost
Interprétabilité	SHAP (TreeExplainer)
Détection d'anomalies	scikit-learn (IsolationForest, DBSCAN, NearestNeighbors)
Visualisation	matplotlib, seaborn
Notebooks	Jupyter, jupytertext (scripts .py appariés aux .ipynb), nbconvert

Structure du dépôt

Le code source complet est organisé en modules réutilisables (*src/load_to_sqlite.py*, *src/credit_risk/features.py*, *src/credit_risk/train.py*), des requêtes SQL versionnées individuellement (*sql/*.sql*), et 5 notebooks d'analyse appariés à leur version script Python via *jupytertext* pour faciliter la revue de code (diff lisible en *.py* plutôt qu'en JSON de notebook). Le dépôt est versionné avec git ; les données brutes et intermédiaires volumineuses (base SQLite de 3.1 Go, fichiers parquet) sont exclues du contrôle de version via *.gitignore* et régénérables via *run_pipeline.py*.

Reproductibilité

L'ensemble du pipeline est paramétré avec une graine aléatoire fixe (*random_state=42*) à chaque étape stochastique (split train/test, validation croisée, SMOTE, Isolation Forest, échantillonnage DBSCAN), garantissant des résultats identiques à chaque nouvelle exécution sur les mêmes données sources.

Rapport généré automatiquement à partir des artefacts produits pendant le développement du projet (notebooks exécutés, fichiers CSV de résultats, figures). Toutes les métriques citées sont issues des sorties réelles des scripts et notebooks du dépôt.